

MySQL 基础语法

1. 数据库操作

创建数据库

```
CREATE DATABASE database_name;
```

删除数据库

```
DROP DATABASE database_name;
```

选择数据库

```
USE database_name;
```

查看所有数据库

```
SHOW DATABASES;
```

2. 表操作

创建表

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints,  
    ...  
);
```

示例:

```
CREATE TABLE students (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50) NOT NULL,  
    age INT,  
    gender CHAR(1),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

删除表

```
DROP TABLE table_name;
```

修改表 (添加列)

```
ALTER TABLE table_name ADD column_name datatype;
```

查看表结构

```
DESC table_name;
```

3. 数据操作 (CRUD)

插入数据 (INSERT)

```
INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
```

示例:

```
INSERT INTO students (name, age, gender) VALUES ('张三', 20, 'M');
```

查询数据 (SELECT)

```
SELECT column1, column2 FROM table_name WHERE condition;
```

示例:

```
SELECT * FROM students;
```

```
SELECT name, age FROM students WHERE age > 18;
```

更新数据 (UPDATE)

```
UPDATE table_name SET column1 = value1 WHERE condition;
```

示例:

```
UPDATE students SET age = 21 WHERE name = '张三';
```

删除数据 (DELETE)

```
DELETE FROM table_name WHERE condition;
```

示例:

```
DELETE FROM students WHERE id = 1;
```

4. 常用子句

WHERE 条件过滤

```
SELECT * FROM students WHERE age >= 18 AND gender = 'M';
```

ORDER BY 排序

```
SELECT * FROM students ORDER BY age DESC; -- 降序
```

```
SELECT * FROM students ORDER BY age ASC; -- 升序 (默认)
```

LIMIT 限制条数

```
SELECT * FROM students LIMIT 5; -- 前5条
```

```
SELECT * FROM students LIMIT 5 OFFSET 2; -- 跳过2条, 取5条
```

GROUP BY 分组

```
SELECT gender, COUNT(*) FROM students GROUP BY gender;
```

HAVING 分组后过滤

```
SELECT gender, COUNT(*) AS cnt FROM students GROUP BY gender HAVING cnt > 1;
```

5. 常用聚合函数

函数	说明
COUNT()	计数
SUM()	求和
AVG()	平均值
MAX()	最大值
MIN()	最小值

示例：

```
SELECT AVG(age) AS avg_age, MAX(age) AS max_age FROM students;
```

6. 多表查询

内连接 (INNER JOIN)

```
SELECT a.name, b.score
FROM students a
INNER JOIN scores b ON a.id = b.student_id;
```

左连接 (LEFT JOIN)

```
SELECT a.name, b.score
FROM students a
LEFT JOIN scores b ON a.id = b.student_id;
```

7. 索引

```
-- 创建索引
CREATE INDEX idx_name ON table_name (column_name);

-- 查看索引
SHOW INDEX FROM table_name;

-- 删除索引
DROP INDEX idx_name ON table_name;
```

 **提示：** SQL 关键字不区分大小写，但通常习惯将关键字大写以提高可读性。每条 SQL 语句以分号 `;` 结尾。

SQL 实战练习：学生成绩管理系统

本教程通过一个完整的**学生成绩管理**案例，演示 MySQL 核心 SQL 语法。包含建表、插入数据、单表查询、多表查询、集合运算等操作。

1. 建表 (DDL)

创建学生信息表

包含学号、姓名、性别、出生年份、院系、家庭住址等字段。

```
CREATE TABLE Student (  
    Id INT(10) AUTO_INCREMENT PRIMARY KEY COMMENT '学号',  
    Name VARCHAR(20) NOT NULL COMMENT '姓名',  
    Sex VARCHAR(4) COMMENT '性别',  
    Birth YEAR COMMENT '出生年份',  
    Department VARCHAR(20) NOT NULL COMMENT '院系',  
    Address VARCHAR(50) COMMENT '家庭住址'  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='学生信息表';
```

创建成绩表

包含编号、学号、课程名、分数等字段，通过 `StuId` 关联学生表。

```
CREATE TABLE Score (  
    Id INT(10) AUTO_INCREMENT PRIMARY KEY COMMENT '编号',  
    StuId INT(10) NOT NULL COMMENT '学号',  
    CName VARCHAR(20) COMMENT '课程名',  
    Grade INT(10) COMMENT '分数'  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='成绩表';
```

2. 插入数据 (DML)

向学生表插入 6 条记录

```
INSERT INTO Student (Id, Name, Sex, Birth, Department, Address)  
VALUES  
    (901, '张老大', '男', 1995, '计算机系', '北京市海淀区'),  
    (902, '张老二', '男', 1996, '中文系', '北京市昌平区'),  
    (903, '张三', '女', 1992, '中文系', '湖南省永州市'),  
    (904, '李四', '男', 1993, '英语系', '辽宁省阜新市'),  
    (905, '王五', '女', 1994, '英语系', '福建省厦门市'),  
    (906, '王六', '男', 1992, '计算机系', '湖南省衡阳市');
```

向成绩表插入 10 条记录

```
INSERT INTO Score (StuId, CName, Grade)  
VALUES  
    (901, '计算机', 98),  
    (901, '英语', 80),  
    (902, '计算机', 65),  
    (902, '中文', 98),  
    (903, '中文', 95),  
    (904, '计算机', 70),  
    (904, '英语', 92),  
    (905, '英语', 94),
```

```
(906, '计算机', 90),  
(906, '英语', 85);
```

3. 单表查询

3.1 查询所有学生信息

```
SELECT * FROM Student;
```

3.2 分页查询

从第 2 条记录开始查询 3 条数据（LIMIT 偏移量，条数）。

```
SELECT * FROM Student  
LIMIT 1, 3;
```

3.3 查询指定列并设置别名

使用 AS 关键字为字段设置中文别名，让结果更易读。

```
SELECT Id AS 学号, Name AS 姓名, Department AS 院系  
FROM Student;
```

3.4 条件查询 (IN)

查询院系为"计算机系"或"英语系"的学生。

```
SELECT * FROM Student  
WHERE Department IN ('计算机系', '英语系');
```

3.5 范围查询 (BETWEEN)

查询出生年份在 2004~2008 年之间的学生。

```
SELECT * FROM Student  
WHERE Birth BETWEEN 2004 AND 2008;
```

3.6 分组统计 (GROUP BY)

按院系统计各院系的学生人数。

```
SELECT Department AS 院系, COUNT(*) AS 人数  
FROM Student  
GROUP BY Department;
```

3.7 聚合函数 — 最高分

查询各科目的最高分。

```
SELECT CName AS 科目, MAX(Grade) AS 最高分  
FROM Score  
GROUP BY CName;
```

3.8 聚合函数 — 平均分

查询各科目的平均成绩，保留两位小数。

```
SELECT
    CName AS 科目,
    ROUND(AVG(Grade), 2) AS 平均成绩
FROM Score
GROUP BY CName;
```

4. 多表查询 (JOIN)

4.1 内连接 (INNER JOIN)

查询"李四"的考试成绩，需关联 `Student` 和 `Score` 两张表。

```
SELECT sc.CName AS 考试科目, sc.Grade AS 考试成绩
FROM Score sc
INNER JOIN Student s ON sc.StuId = s.Id
WHERE s.Name = '李四';
```

4.2 左连接 (LEFT JOIN)

查询所有学生及其考试成绩。未参加考试的学生也会显示（成绩字段为 NULL）。

```
SELECT s.*, sc.CName, sc.Grade
FROM Student s
LEFT JOIN Score sc ON s.Id = sc.StuId;
```

4.3 多表分组统计

查询每位学生的总成绩。

```
SELECT
    s.Id AS 学号,
    s.Name AS 姓名,
    SUM(sc.Grade) AS 总成绩
FROM Student s
INNER JOIN Score sc ON s.Id = sc.StuId
GROUP BY s.Id, s.Name;
```

4.4 多表条件过滤

查询计算机成绩低于 95 分的学生信息。

```
SELECT s.*
FROM Student s
INNER JOIN Score sc ON s.Id = sc.StuId
WHERE sc.CName = '计算机' AND sc.Grade < 95;
```

4.5 多表查询 + 排序

查询计算机成绩，按分数从高到低排序。

```
SELECT
    s.Name AS 姓名,
    sc.Grade AS 计算机成绩
FROM Score sc
```

```
INNER JOIN Student s ON sc.StuId = s.Id
WHERE sc.CName = '计算机'
ORDER BY sc.Grade DESC;
```

4.6 综合查询

查询地址中含有"湖南"的学生，计算其年龄并显示成绩详情。

```
SELECT
    s.Name AS 姓名,
    (2026 - s.Birth) AS 年龄,
    s.Department AS 院系,
    sc.CName AS 考试科目,
    sc.Grade AS 成绩
FROM Student s
INNER JOIN Score sc ON s.Id = sc.StuId
WHERE s.Address LIKE '%湖南%';
```

5. 集合运算 (UNION)

UNION 并集

查询所有在**学生表**或**成绩表**中出现过的学号，自动去重。

```
SELECT Id AS StuId FROM Student
UNION
SELECT StuId FROM Score;
```

 **提示：** 建议按顺序在 MySQL 环境中逐条执行以上 SQL，观察每一步的输出结果，加深理解。